



Transport layer

Dr. Alekha Kumar Mishra



Process to process delivery

- Datalink layer – node-to-node delivery
- Network layer – host-to-host delivery
- Transport layer – process-to-process delivery



Client/Server Paradigm

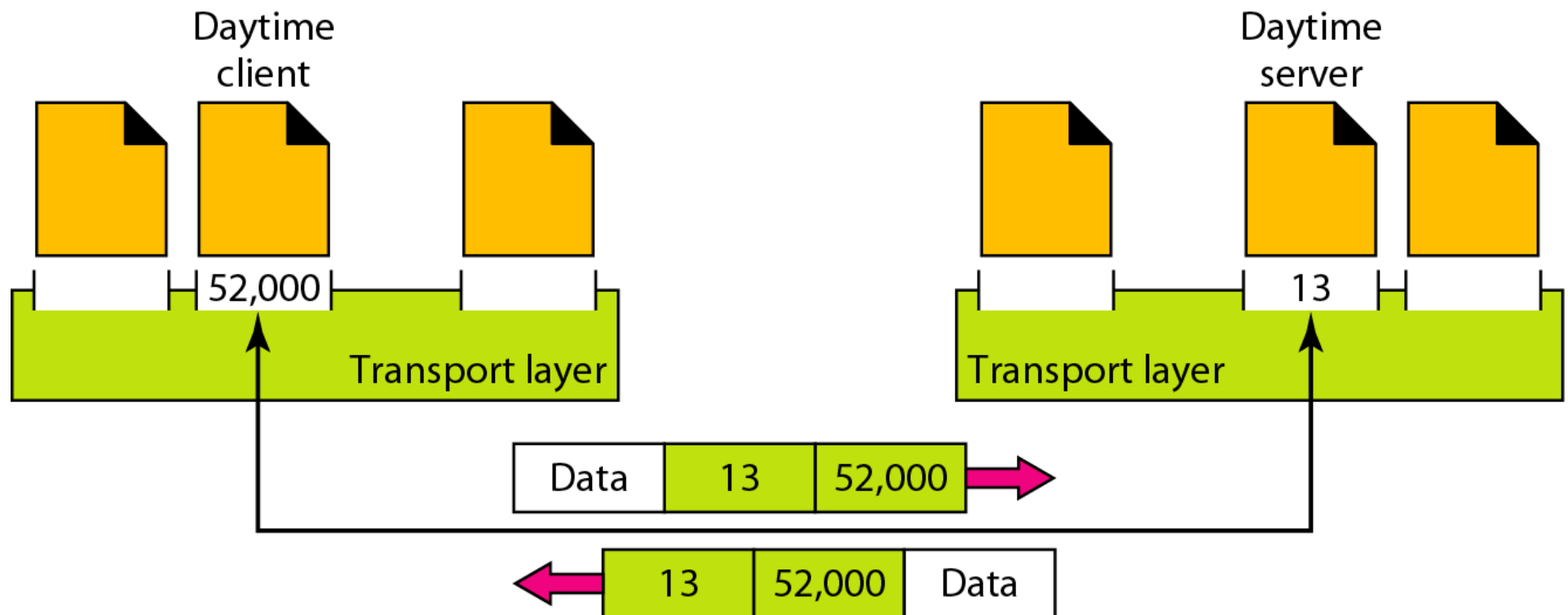
- A process on the local host, called a **client**
- Client needs services from a process usually on the remote host, called a **server**
- Both processes (client and server) have the same name
 - Chat client
 - Chat Server



Transport Layer Address

- A transport layer address, called a **port number**, to choose among multiple processes running on the destination host
- The destination port number is needed for delivery
- The source port number is needed for the ____.
- In the Internet model, the port numbers are **16-bit integers**
- The client program port number is **randomly** by the transport layer software running on the client host (The **ephemeral** port number)
- The server process must also define itself with a port number (cannot be chosen randomly)
- The Internet has decided to use universal port numbers for servers (called **well-known port numbers**)

Port number



IANA (Internet Assigned Number Authority) ranges

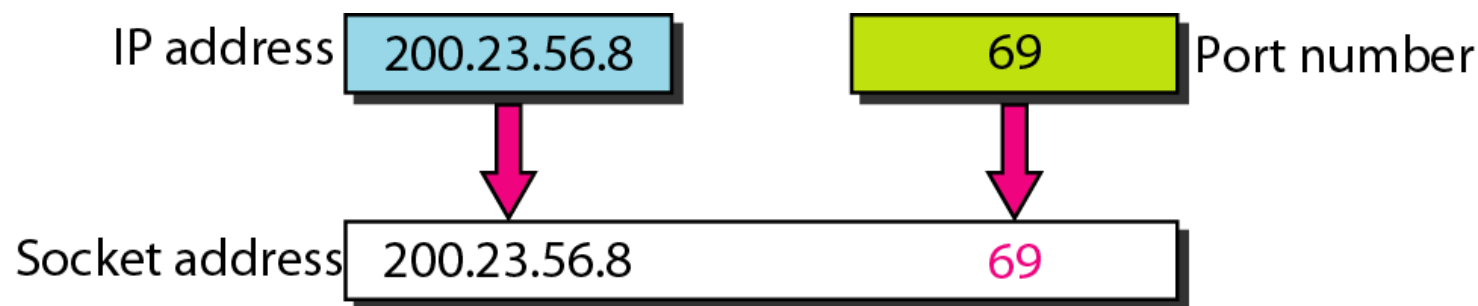


- **Well-known ports** – 0 to 1023 (assigned and controlled)
- **Registered ports** – 1024 to 49,151 (not assigned or controlled)
- **Dynamic ports** – 49,152 to 65,535 (neither controlled nor registered) the ephemeral ports

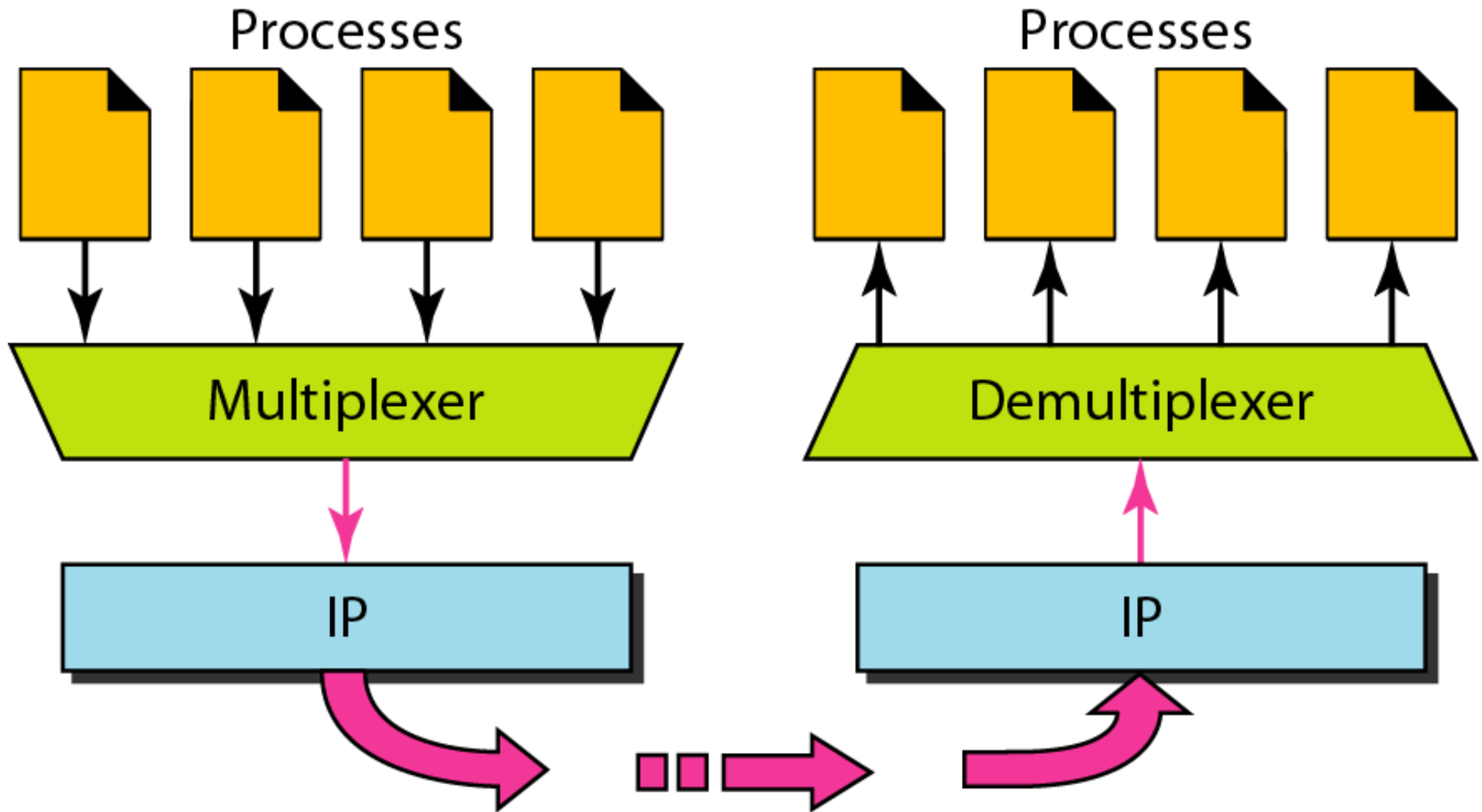


Socket Addresses

- Process-to-process delivery needs two identifiers, IP address and the port number, at each end to make a **connection**
- The combination of an IP address and a port number is called a **socket address**
- A transport layer protocol needs a pair of socket addresses
 - The IP header contains the IP addresses; the UDP or TCP header contains the port numbers.



Multiplexing and Demultiplexing at Transport Layer



Connectionless Versus Connection-Oriented Service



- In a connectionless service, the packets are sent from one party to another with **no need** for connection establishment or connection release
 - They may be **delayed** or **lost** or may arrive **out of sequence**
 - There is **no acknowledgment** either
- In a connection-oriented service, a connection is first **established** between the sender and the receiver
 - Data are transferred in order
 - At the end, the connection is **released**

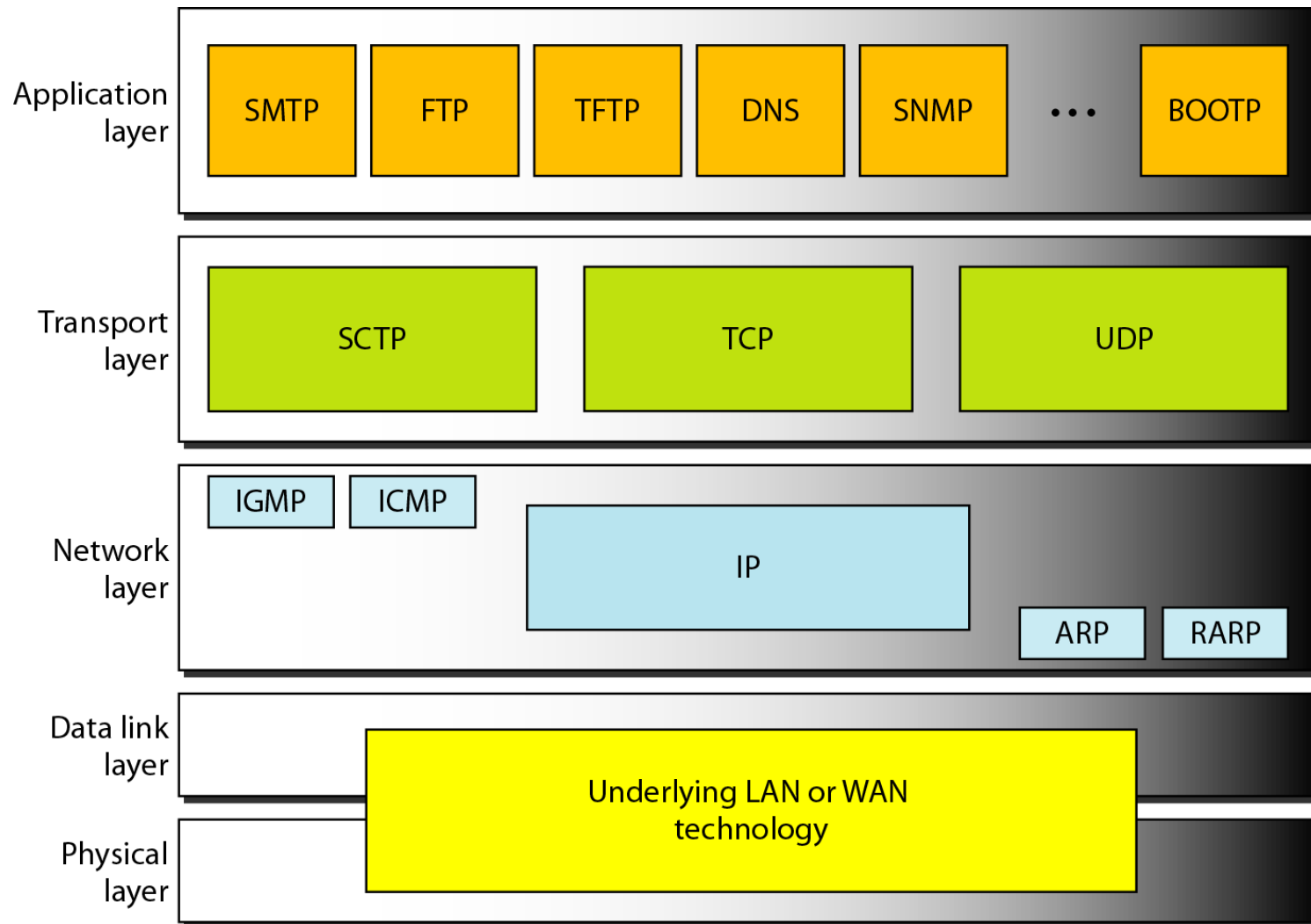


Reliable vs. Unreliable

- Reliable transport layer protocols **implement** flow and error control at the transport layer
- Unreliable transport layer protocols focus on **fast service**, and does not provided flow and error control (real-time applications)
- In the Internet, there are three common different transport layer protocols
 - **UDP** is connectionless and unreliable
 - **TCP** and **SCTP** are connection-oriented and reliable



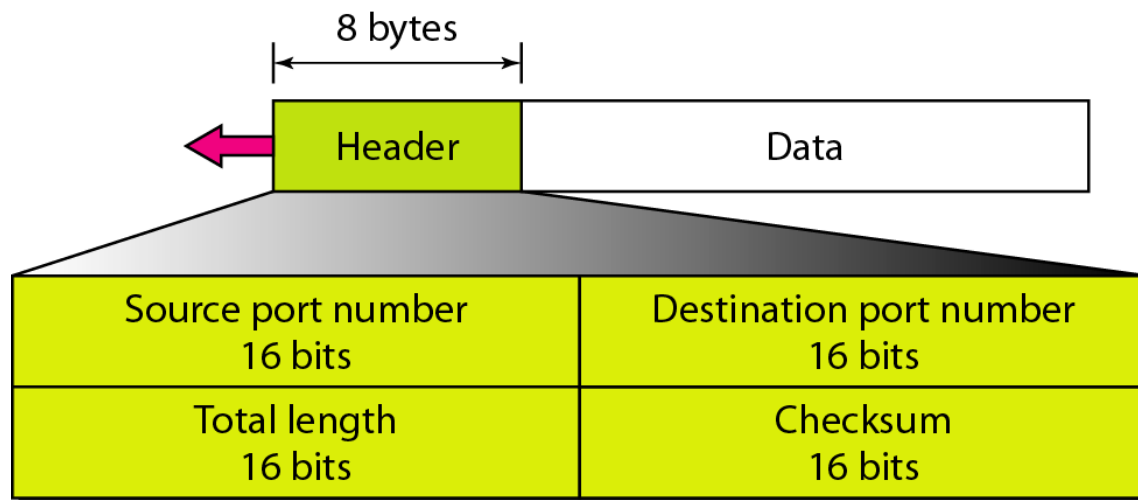
Transport Layer Protocols





USER DATAGRAM PROTOCOL (UDP)

- A connectionless, unreliable transport protocol
- It performs very limited error checking
- Suitable choice for a process that wants to send a small message and **does not care** much about reliability
- $\text{UDP length} = \text{IP length} - \text{IP header's length}$



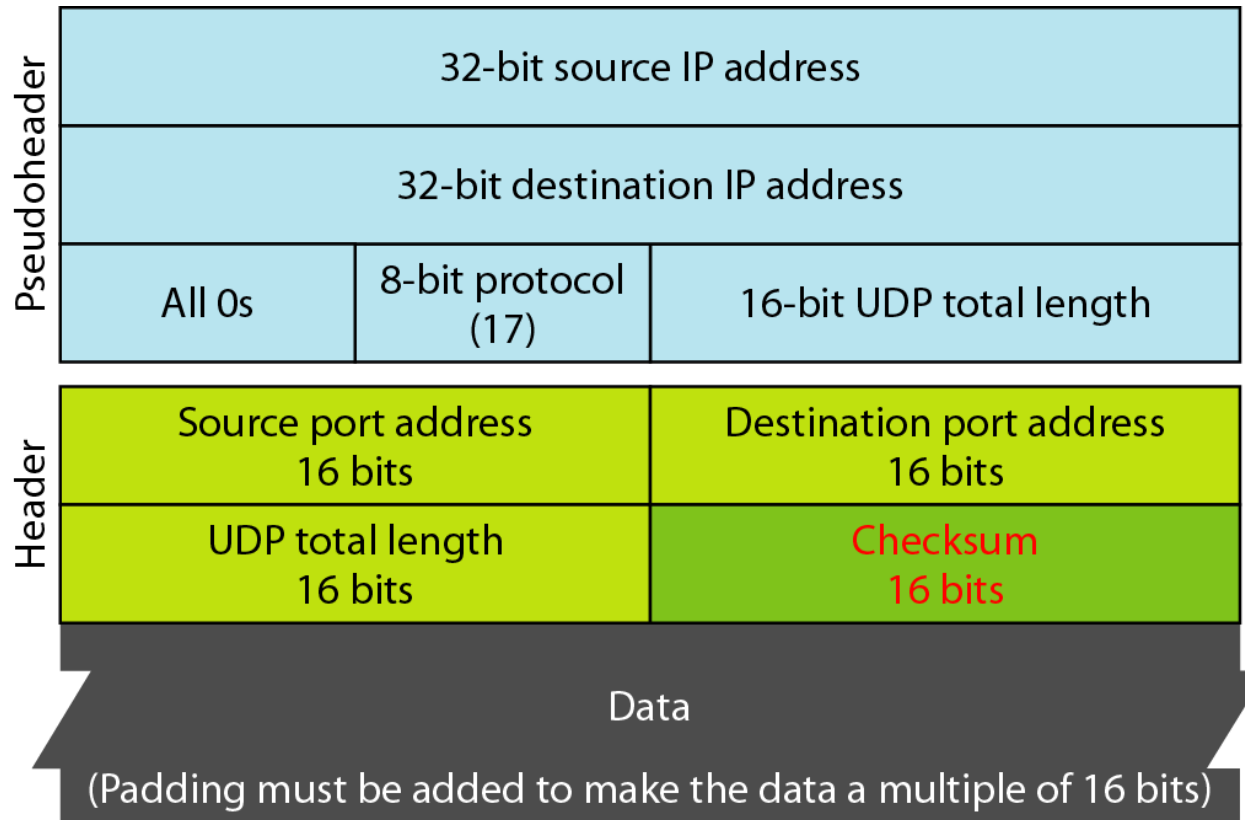


Well known ports of UDP

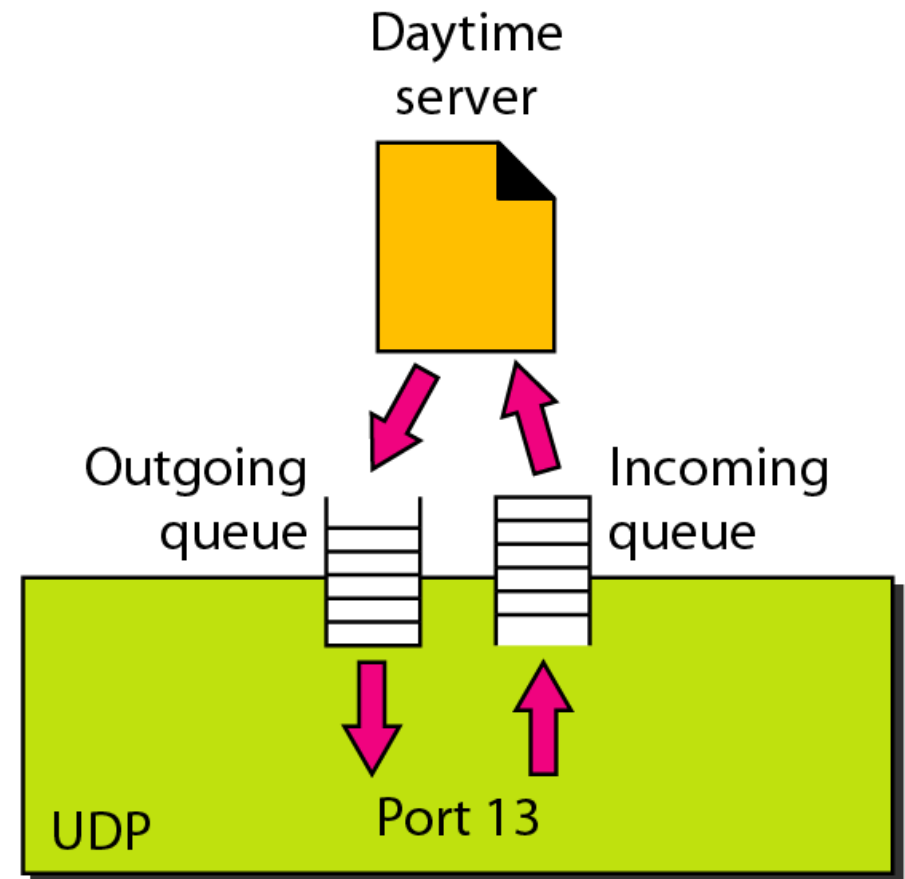
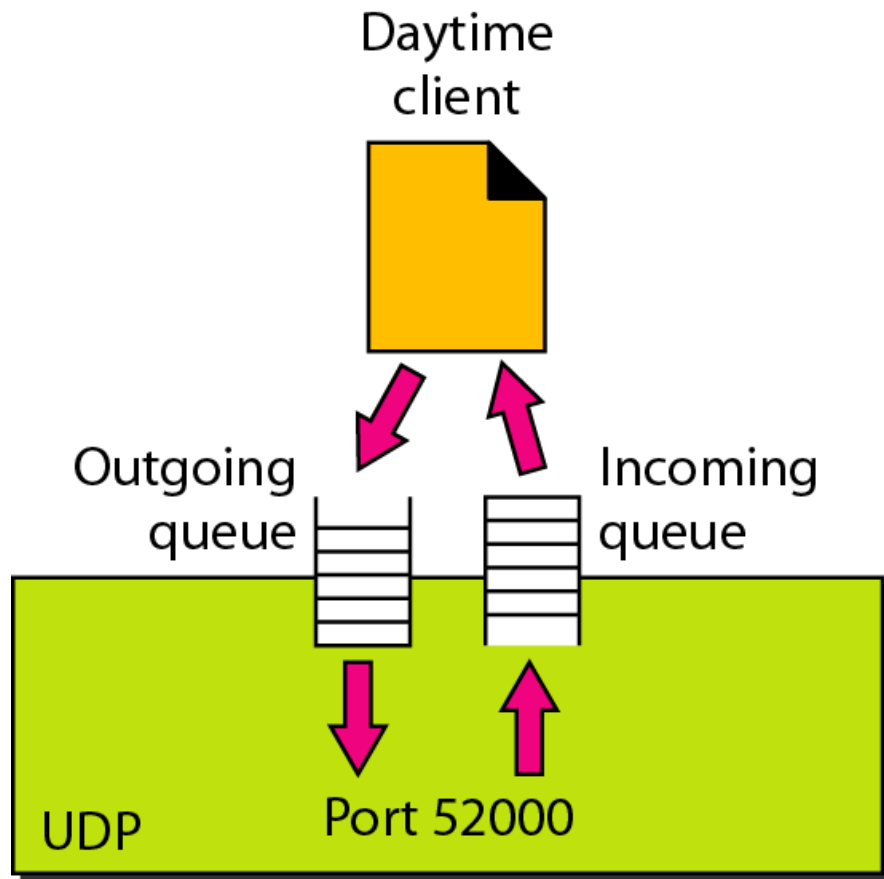
<i>Port</i>	<i>Protocol</i>	<i>Description</i>
7	Echo	Echoes a received datagram back to the sender
9	Discard	Discards any datagram that is received
11	Users	Active users
13	Daytime	Returns the date and the time
17	Quote	Returns a quote of the day
19	Chargen	Returns a string of characters
53	Nameserver	Domain Name Service
67	BOOTPs	Server port to download bootstrap information
68	BOOTPc	Client port to download bootstrap information
69	TFTP	Trivial File Transfer Protocol
111	RPC	Remote Procedure Call
123	NTP	Network Time Protocol
161	SNMP	Simple Network Management Protocol
162	SNMP	Simple Network Management Protocol (trap)



Checksum calculation in UDP



Queuing





Use of UDP

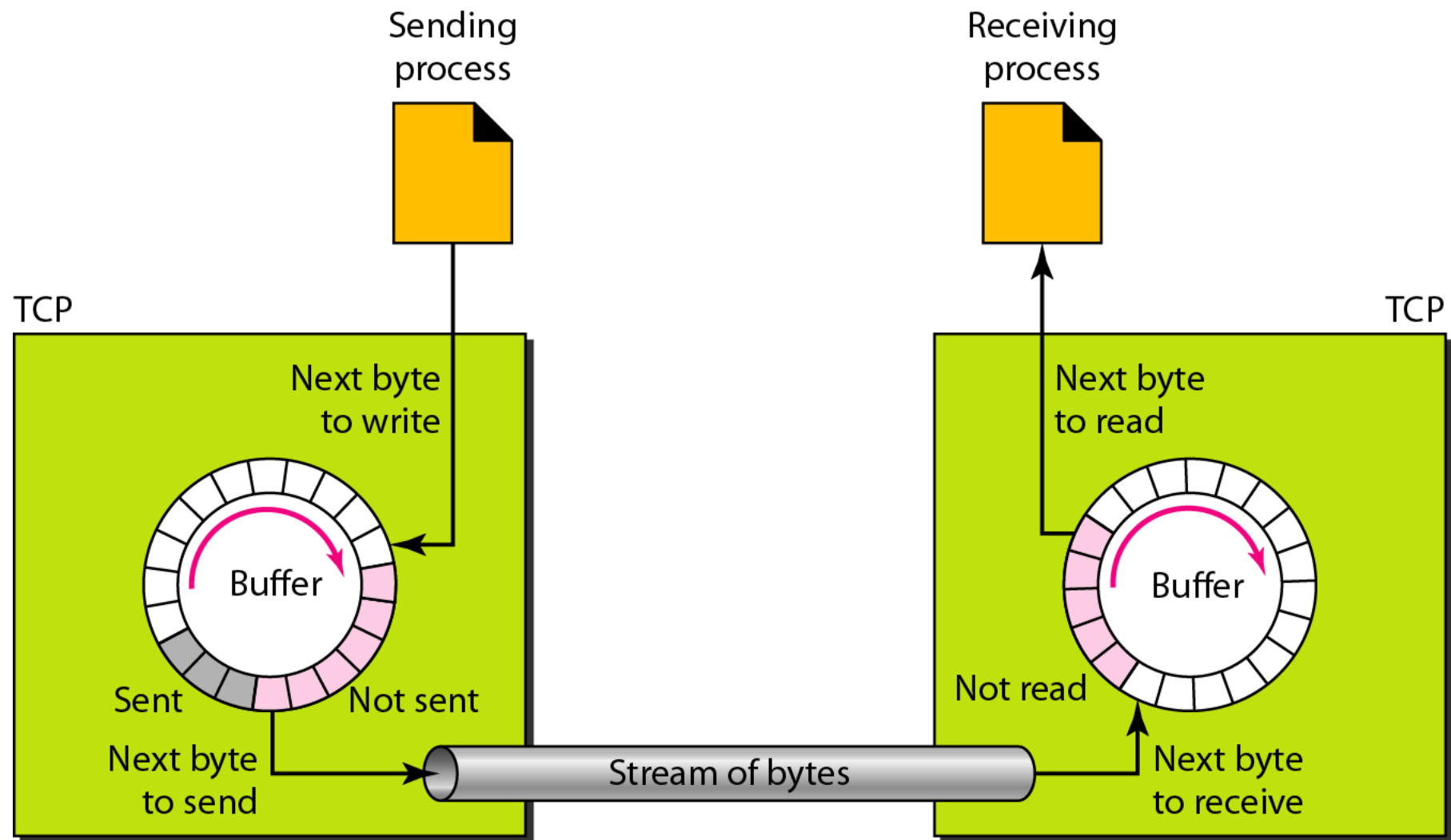
- Suitable for a process that requires simple request-response communication
- No requirement of flow and error control
- Not suitable for sending bulk data
- Suitable for multicasting
- UDP is used for management processes such as SNMP
- UDP is used for RIP



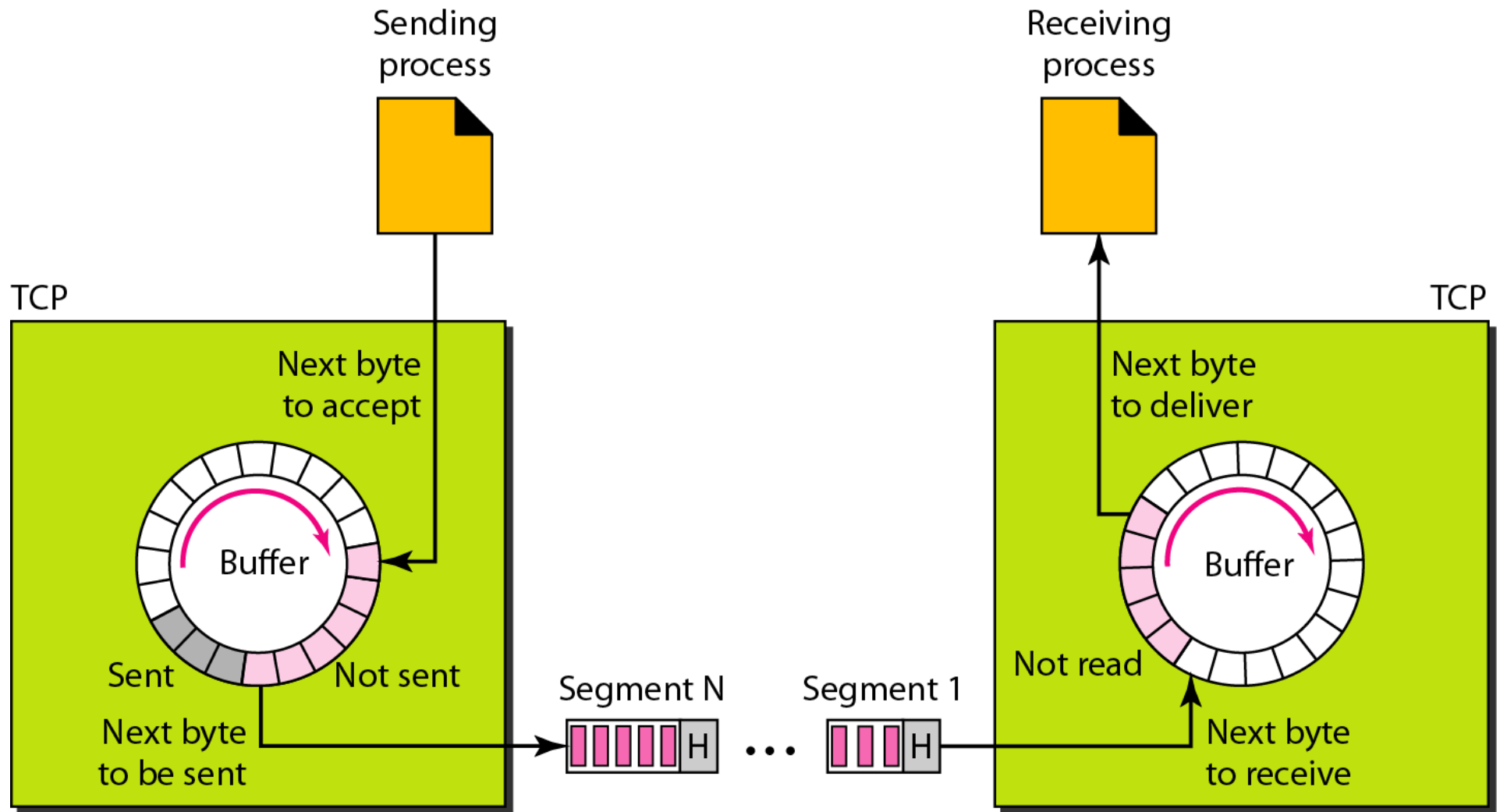
Transmission Control Protocol (TCP)

- Process-to-Process Communication Service
- Stream Delivery Service
 - Allows the sending process to deliver data as a **stream of bytes** and the receiving process to obtain data as a stream of bytes

Sending and Receiving Buffer

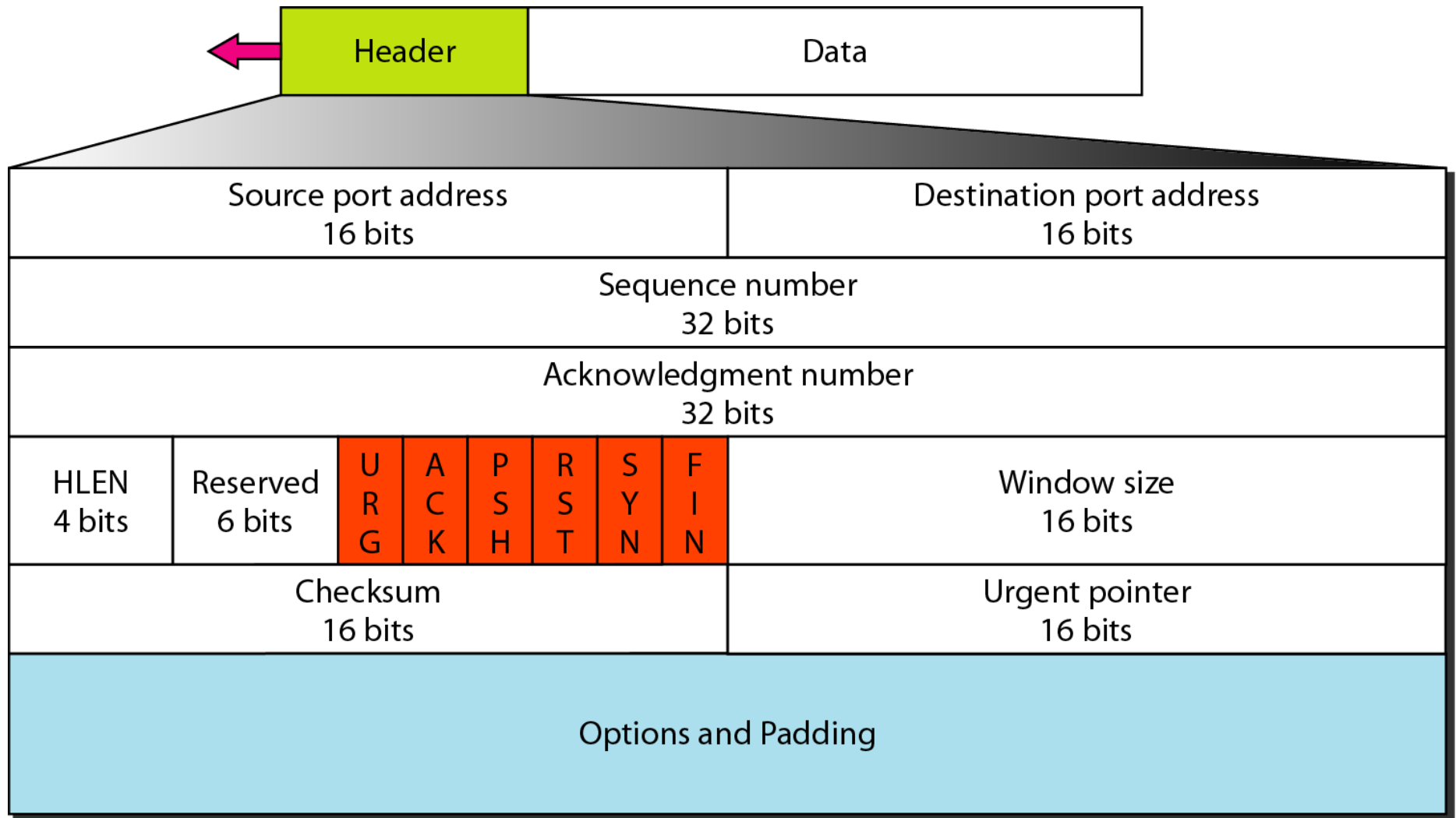


TCP Segment





TCP Segment format





TCP Segment format

- The segment consists of a 20- to 60-byte header+ data from the application program
 - The header size > 20 bytes when it contains options
- Source port address (16-bit) - Defines the port number of the application in the hos
- Destination port address (16-bit) - Defines the port number of the application in the receiving host
- Sequence number (32-bit) - defines the number assigned to the first byte of data in this segment
- ACK number(32-bit) - defines the byte number that the receiver of the segment is expecting to receive from the other party
 - If byte number x received successfully , it defines $x + 1$ as the ACK number
- Header length(4-bit) - the number of 4-byte words in the header
 - Value can be between 5 ($5 \times 4 = 20$) and 15 ($15 \times 4 = 60$).
- Reserved (6-bit) - for future use.



TCP Segment format

<i>Flag</i>	<i>Description</i>
URG	The value of the urgent pointer field is valid.
ACK	The value of the acknowledgment field is valid.
PSH	Push the data.
RST	Reset the connection.
SYN	Synchronize sequence numbers during connection.
FIN	Terminate the connection.

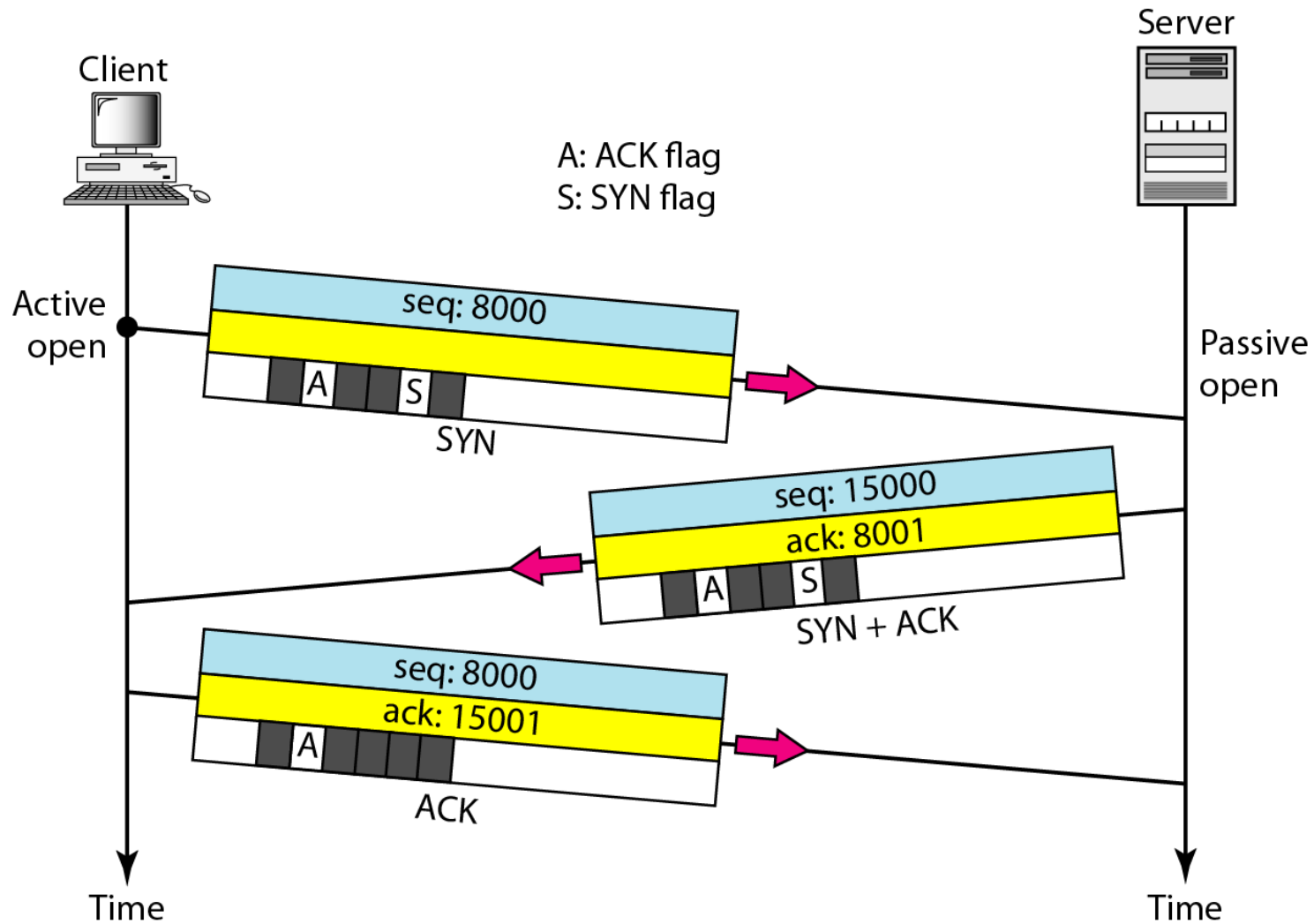
- Control (6 bit) - control bits or flags
 - enables flow control, connection establishment and termination, connection abortion, and the mode of data transfer in TCP
- Window size - This field defines the size of the window, in bytes, that the other party must maintain
 - Normally referred to as the receiving window (rwnd) and is determined by the receiver
- Checksum(16-bit) - Contains the checksum
 - The inclusion of the checksum field for TCP checksum calculation is mandatory
- Urgent pointer(16-bit) - valid only if the urgent flag is set
 - Used when the segment contains urgent data
- Options(up to 40 bytes) - optional information in the TCP header.



TCP Connection

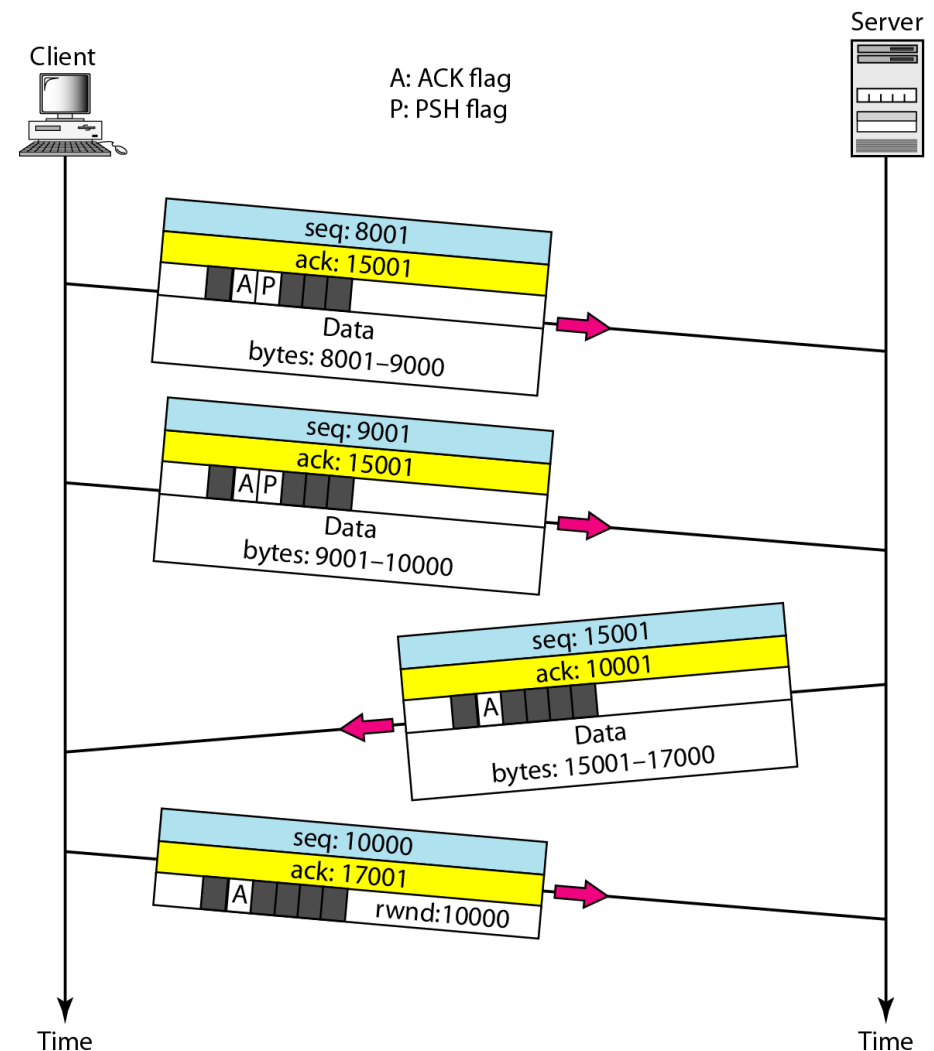
- Establishes a virtual path between the source and destination
- All the segments belonging to a message are then sent over this virtual path
- TCP uses the services of IP to deliver individual segments to the receiver, while controls the connection
- If a segment is lost or corrupted, it is retransmitted
- If a segment arrives out of order, TCP holds it until the missing segments arrive
- Three phases
 - Connection establishment
 - Data transfer, and
 - Connection termination

TCP Connection Establishment : Three-way handshaking

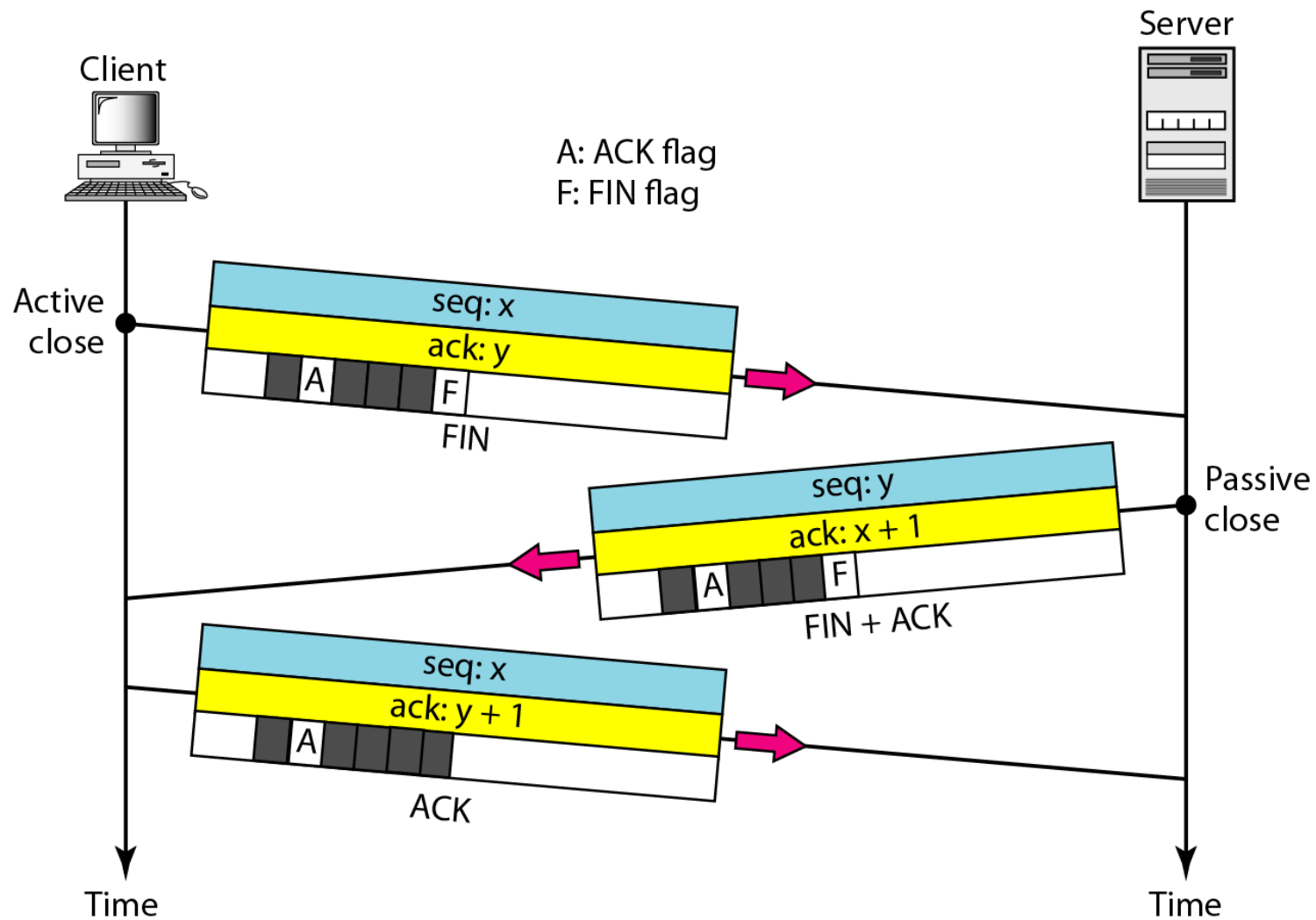


Data transfer phase

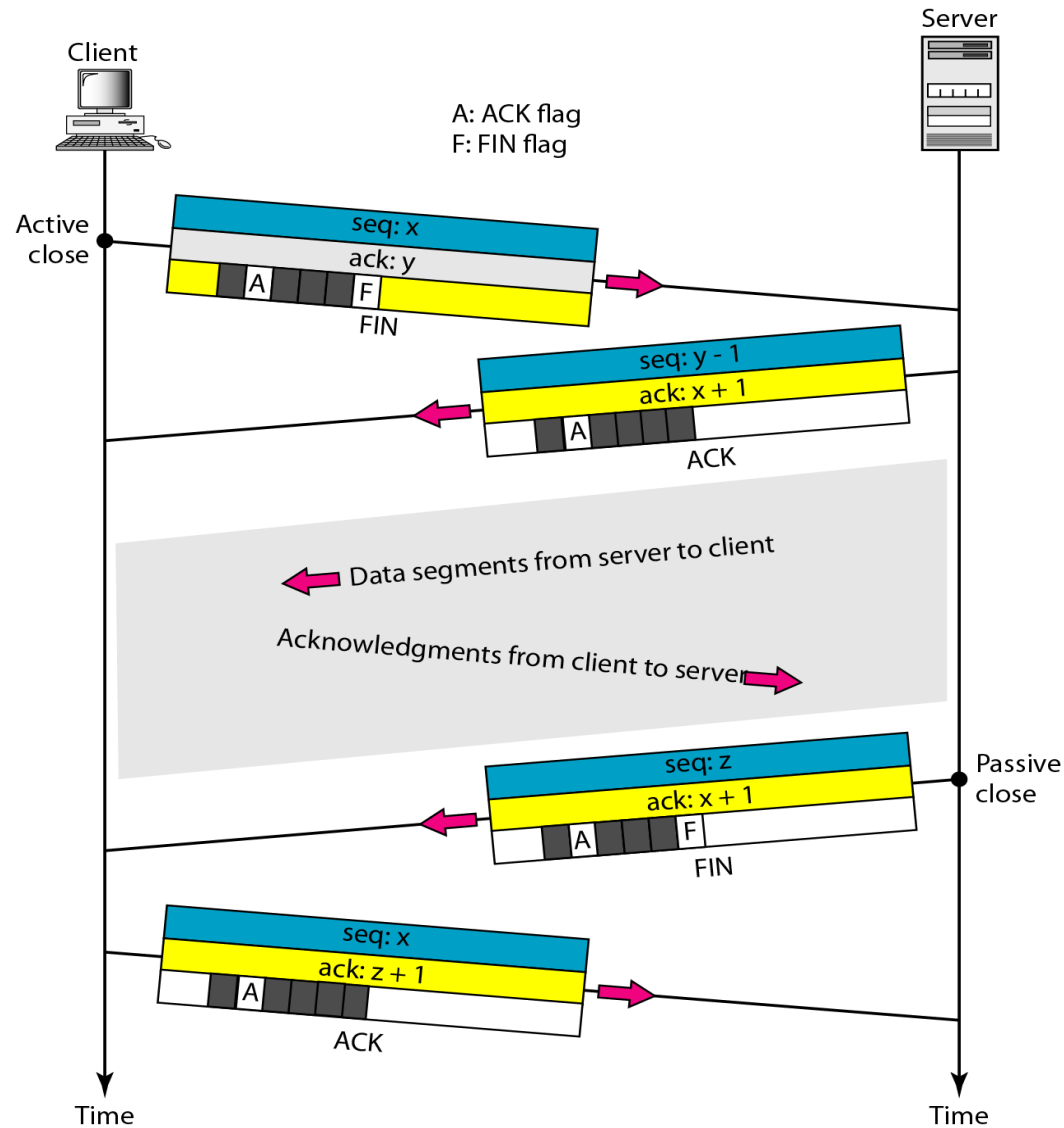
- The client and server can both send data and acknowledgments
- The acknowledgment is piggybacked with the data
- The data segments sent by the client have the PSH (push) flag set so that the server TCP knows to deliver data to the server process as soon as they are received



Connection Termination : Three-Way Handshaking



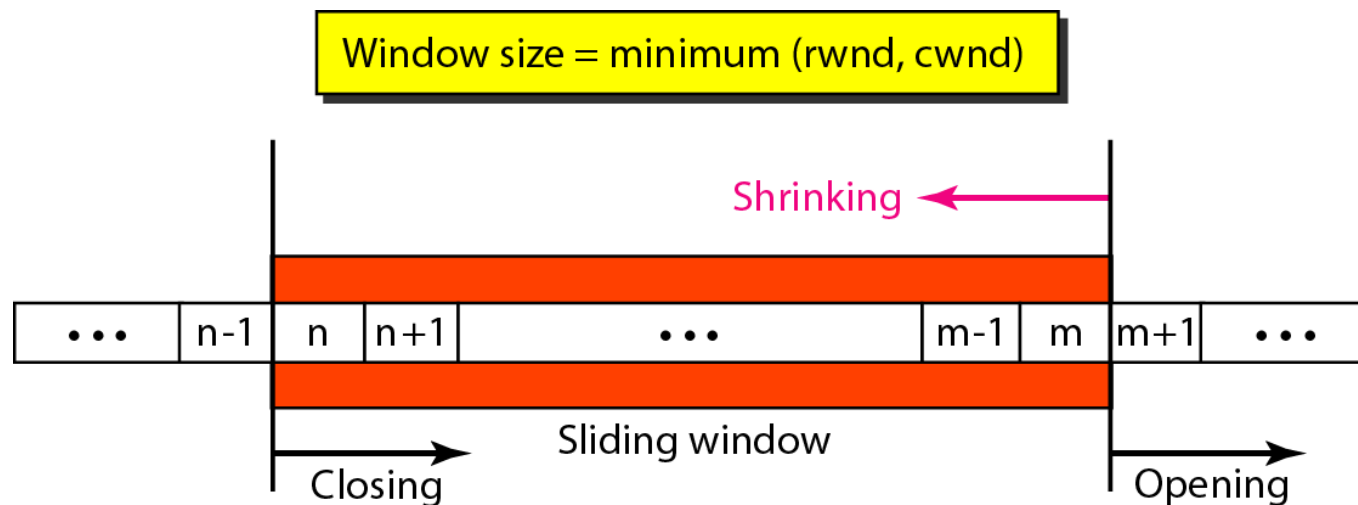
Connection Termination : Half Close





Flow Control

- The sliding window protocol used by TCP is something between the Go-Back-N and Selective Repeat

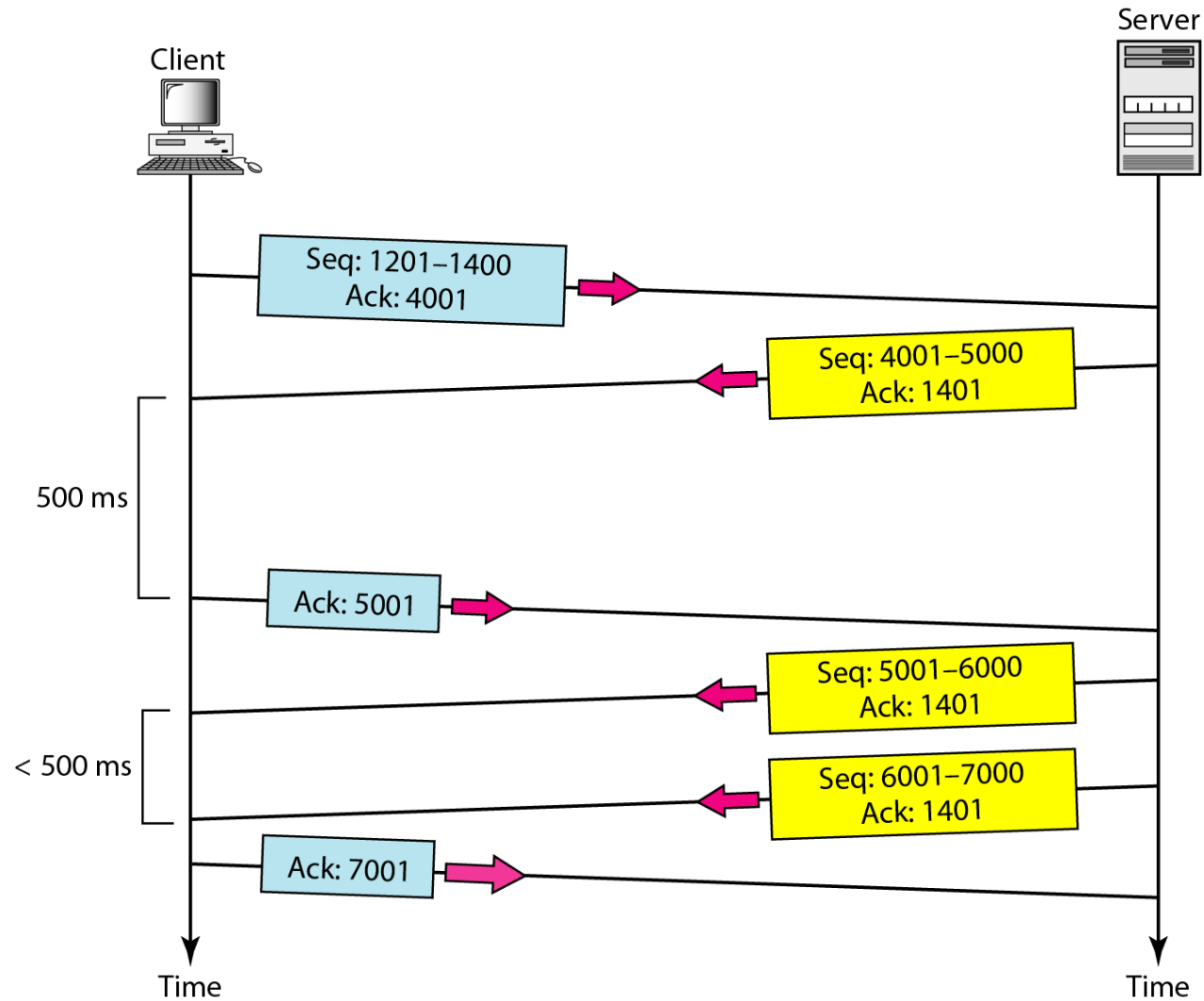




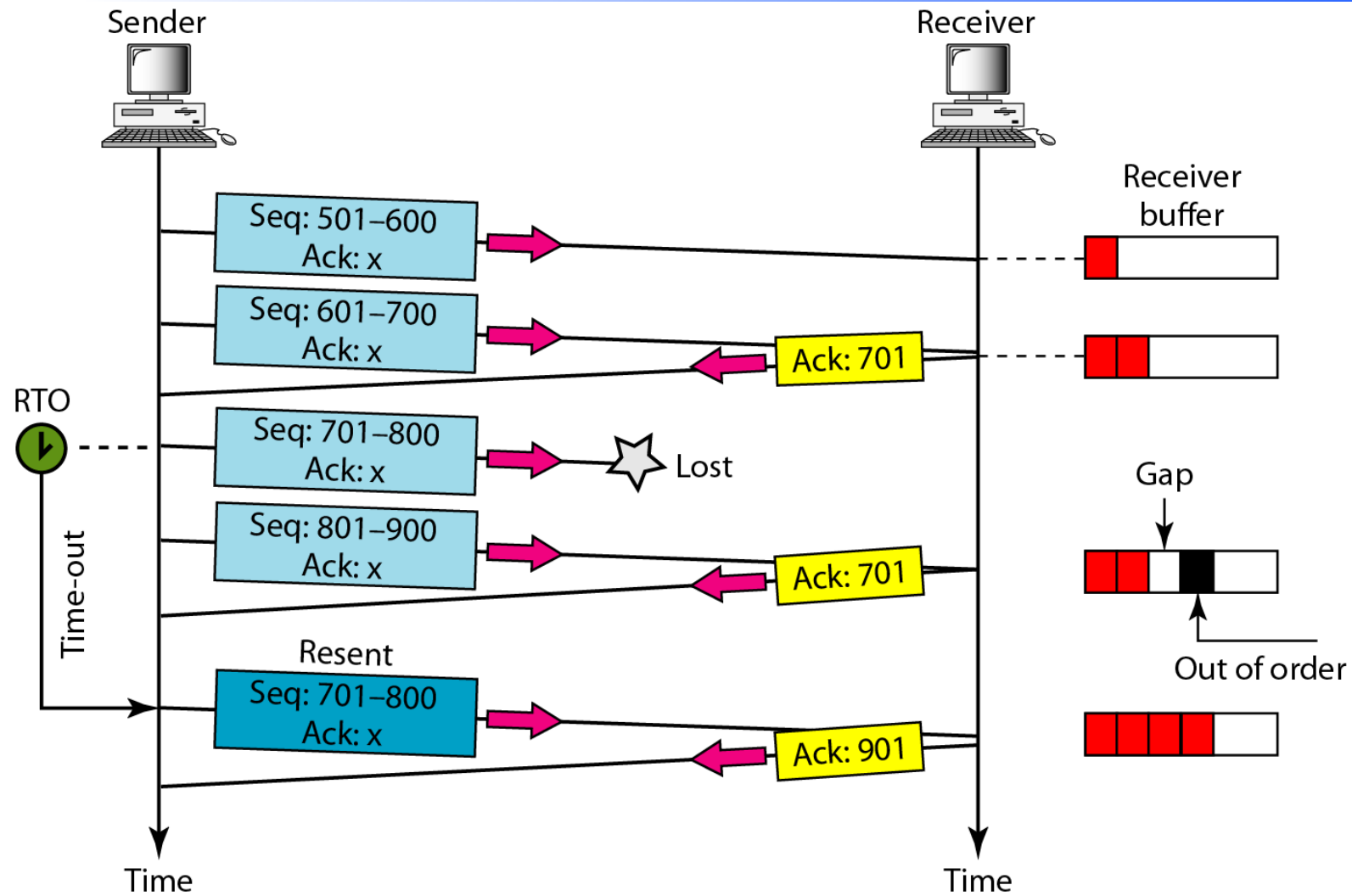
Error Control

- Checksum
 - a checksum field which is used to check for a corrupted segment
- Acknowledgment
 - TCP uses acknowledgments to confirm the receipt of data segments
- Retransmission
 - The heart of the error control mechanism is the retransmission of segments. When a segment is corrupted, lost, or delayed, it is retransmitted

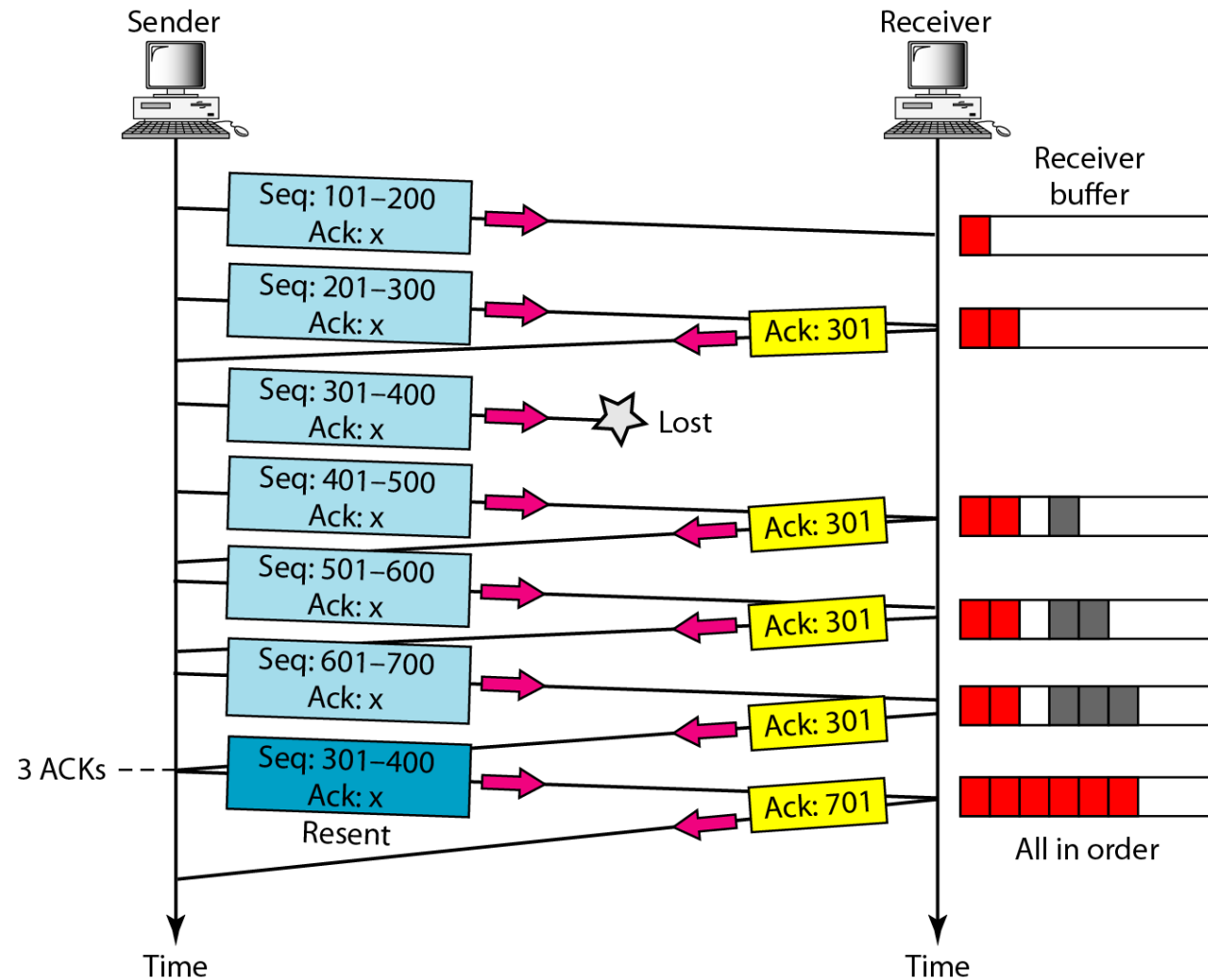
Normal scenario



Segment is lost



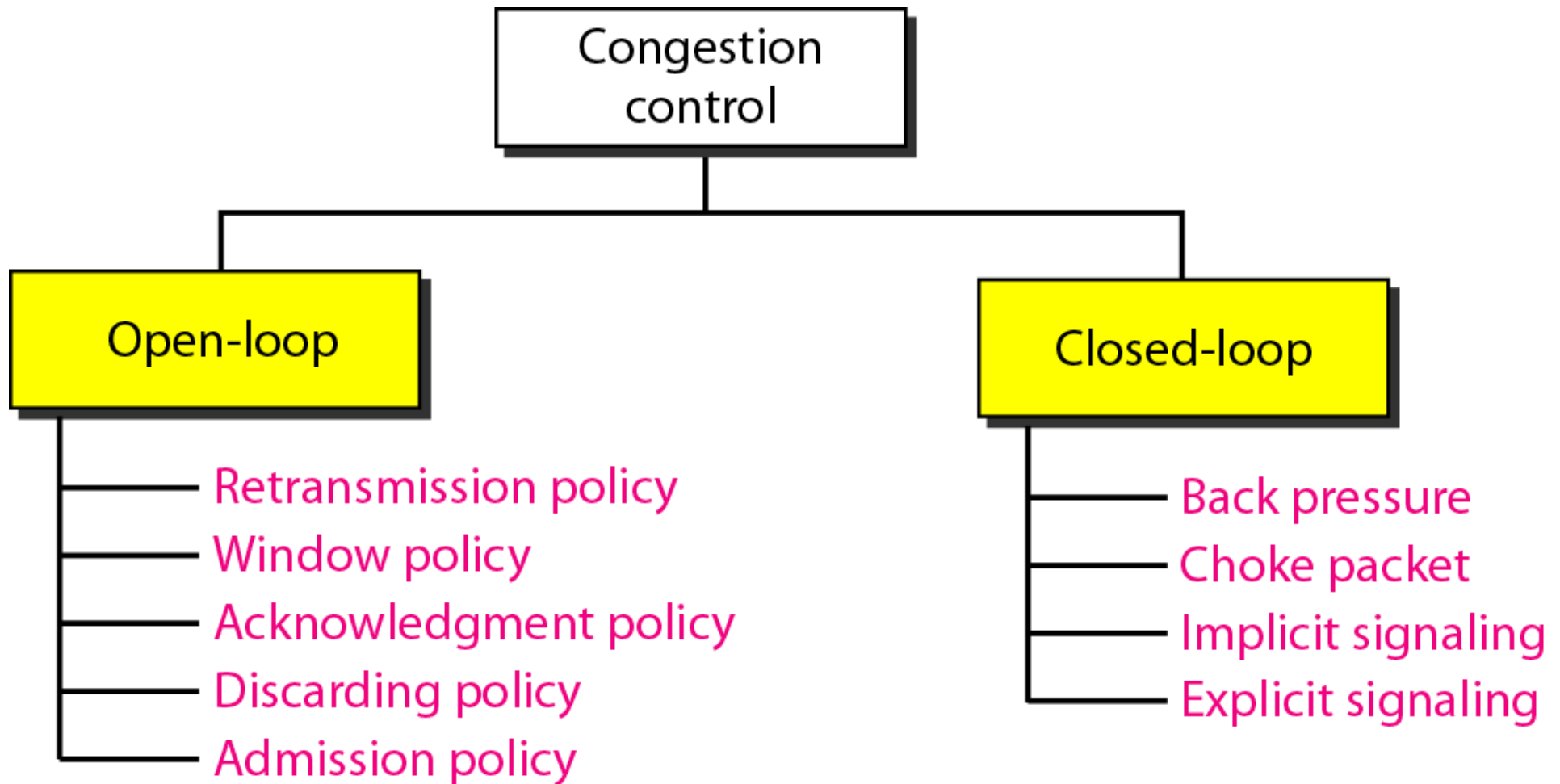
Fast Retransmission





Congestion control

- Congestion in a network may occur if the load on the network is high (the number of packets sent to the network $>$ the capacity of the network)
- Congestion control refers to keep the load below the capacity.
- Congestion control techniques and mechanisms can either prevent congestion, before it happens, or remove congestion, after it has happened





Closed-Loop Congestion Control

- Backpressure
 - A congested node stops receiving data from the immediate upstream node or nodes
 - Propagates, in the opposite direction of data flow, to the source
- Choke Packet
 - A choke packet is a packet sent by a node to the source to inform it of congestion.
 - The choke packet is the warning from the router , which has encountered congestion, to the source station directly
- Implicit Signaling
 - There is no communication between the congested node or nodes and the source. The source guesses that there is a congestion somewhere in the network from other symptoms
- Backward Signaling
 - Setting a bit in a packet moving in opposite direction of congestion can warn the source to slow down to avoid the discarding of packets.
- Forward Signaling
 - Set a bit in a packet moving in the direction of the congestion to warn the destination that there is congestion



Congestion Policy

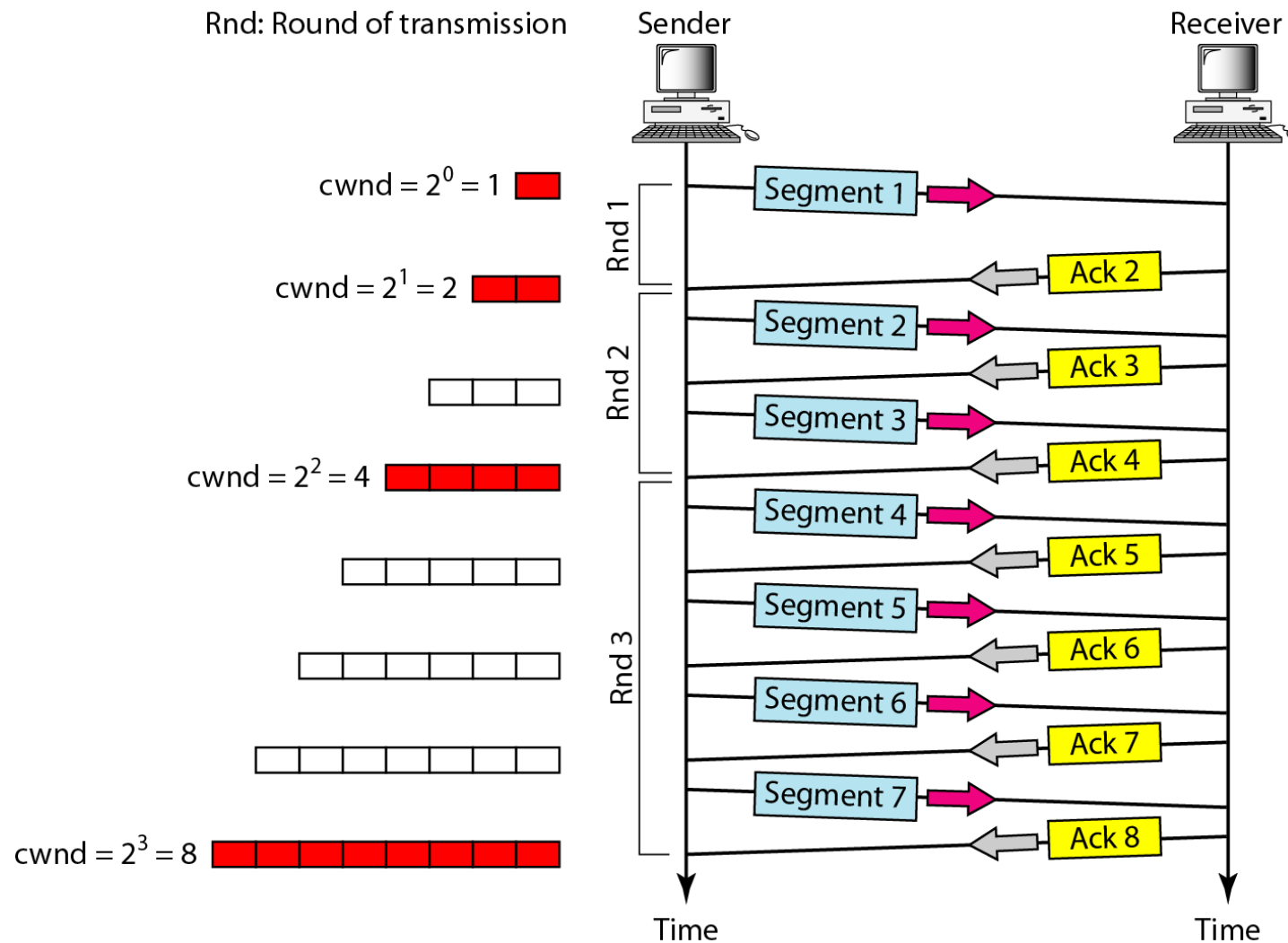
- TCP's general policy for handling congestion is based on three phases:
 - slow start,
 - congestion avoidance, and
 - congestion detection



Slow start

- The sender starts with a very slow rate of transmission, but increases the rate rapidly to reach a threshold.
- When the threshold is reached, the data rate is reduced to avoid congestion
- Finally if congestion is detected, the sender goes back to the slow-start or congestion avoidance phase
- **Slow Start: Exponential Increase**
 - This algorithm is based on the idea that the size of the congestion window (cwnd) starts with one maximum segment size (MSS)
 - The size of the window increases one MSS each time an acknowledgment is received
 - The window starts slowly, but grows exponentially

• Slow Start: Exponential Increase

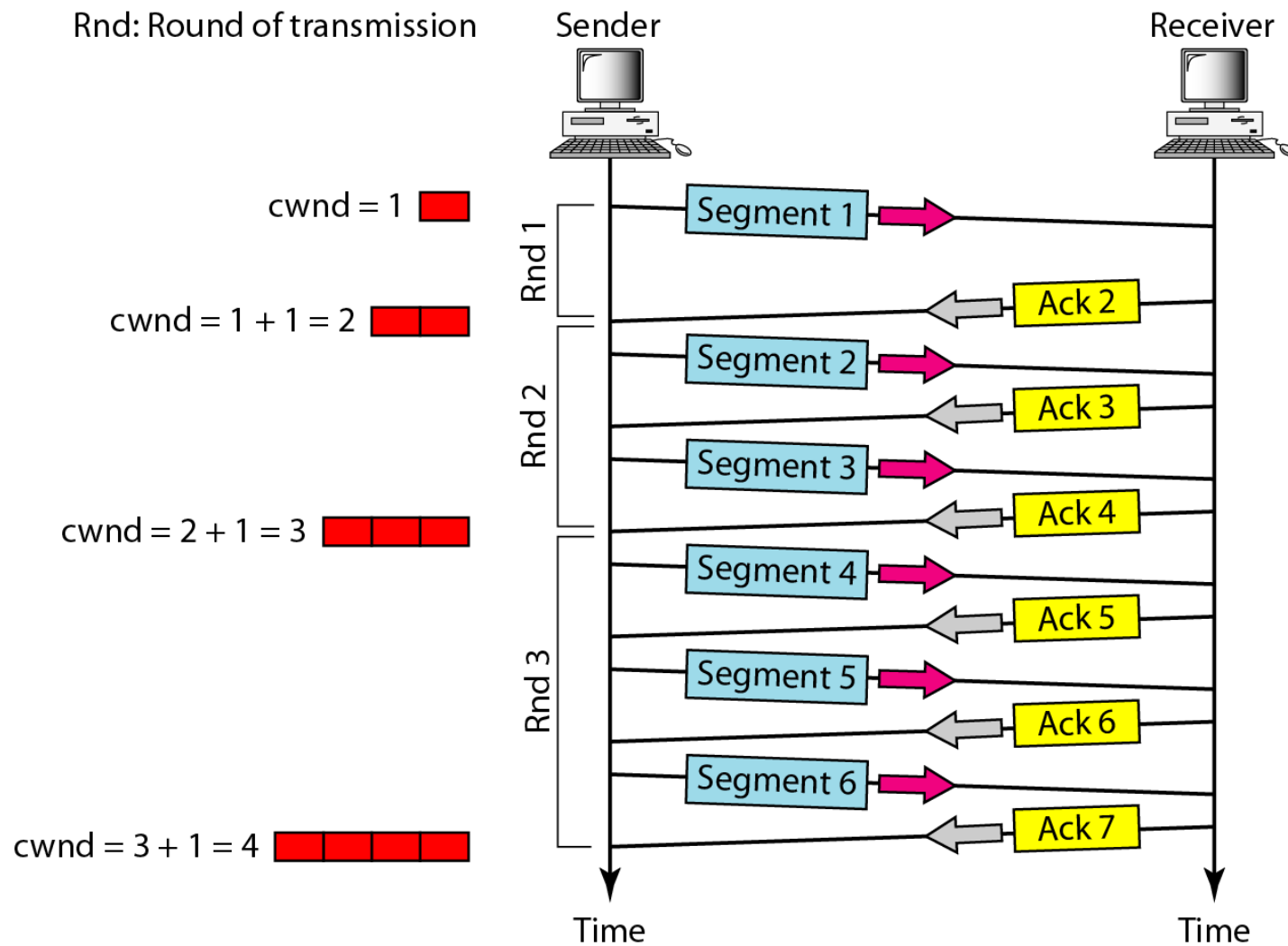




Congestion Avoidance: Additive Increase

- To avoid congestion before it happens, one must slow down this exponential growth
- TCP defines another algorithm called congestion avoidance, which undergoes an additive increase instead of an exponential one
- When the size of the congestion window reaches the slow-start threshold, the slow-start phase stops and the additive phase begins
- In this algorithm, each time the whole window of segments is acknowledged (one round), the size of the congestion window is increased by 1

Congestion Avoidance: Additive Increase



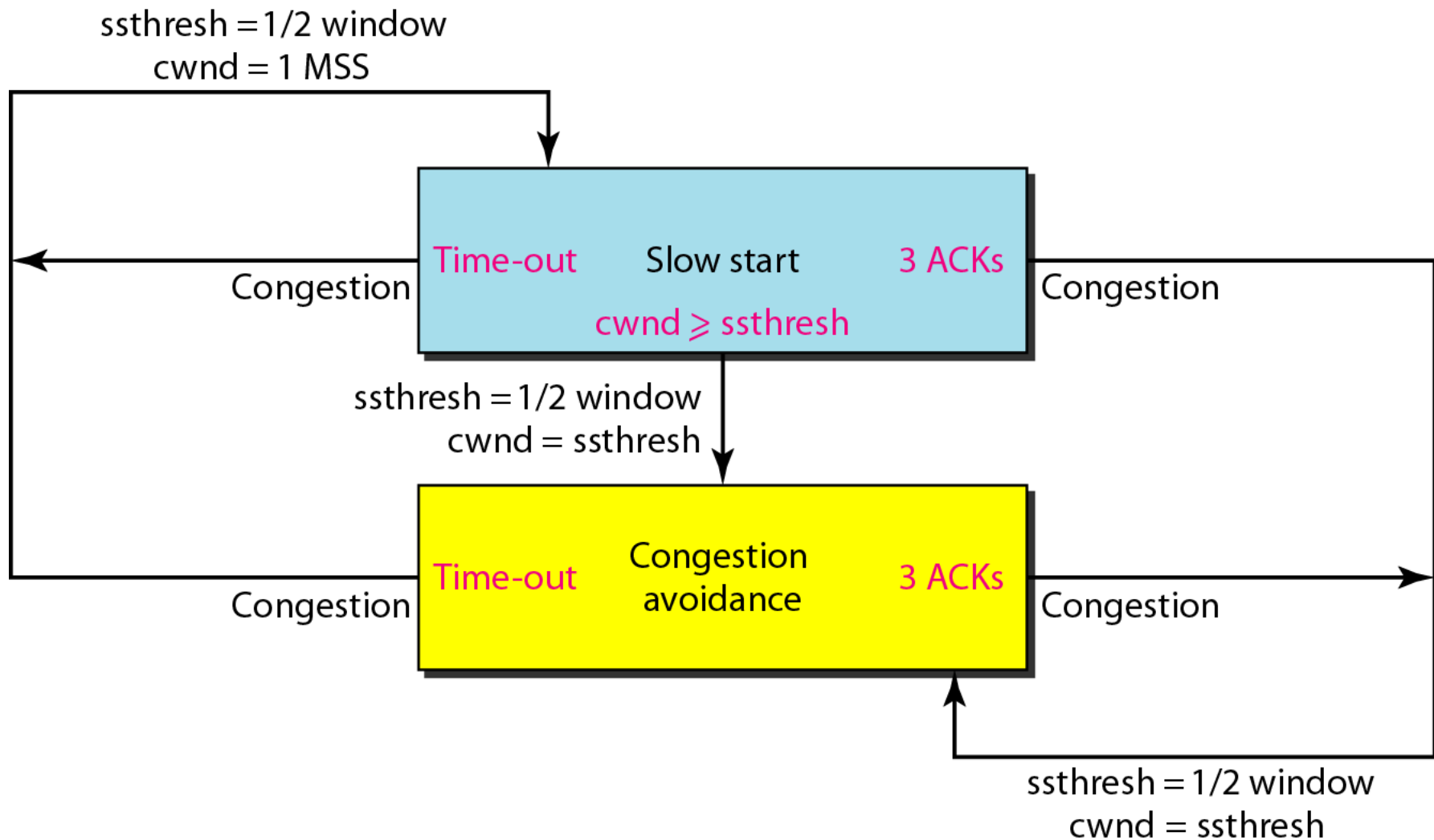


• Congestion Detection: Multiplicative Decrease

- If congestion occurs, the congestion window size must be decreased
- TCP reacts strongly:
 - It sets the value of the threshold to one-half of the current window size.
 - It sets cwnd to the size of one segment.
 - It starts the slow-start phase again.
- If three ACKs are received, there is a weaker possibility of congestion(fast transmission and fast recovery), TCP has a weaker reaction:
 - It sets the value of the threshold to one-half of the current window size.
 - It sets cwnd to the value of the threshold
 - It starts the congestion avoidance phase.



TCP congestion policy summary





Quality of service (QoS)

- It is the objective of the data flow
- Flow characteristics
 - Reliability
 - Delay
 - Jitter
 - Bandwidth



Techniques to improve QoS

- Packet Scheduling
 - FIFO queuing
 - Priority queuing
 - Weighted Fair Queuing

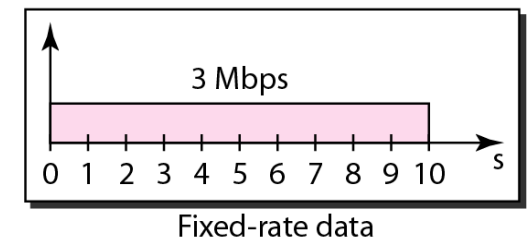
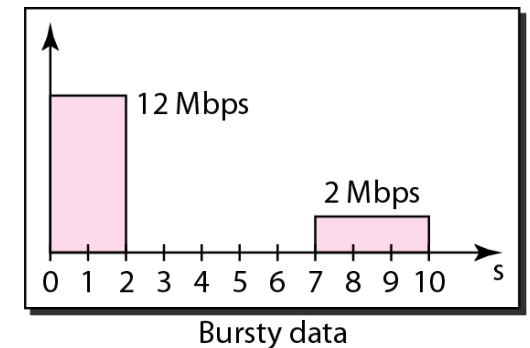
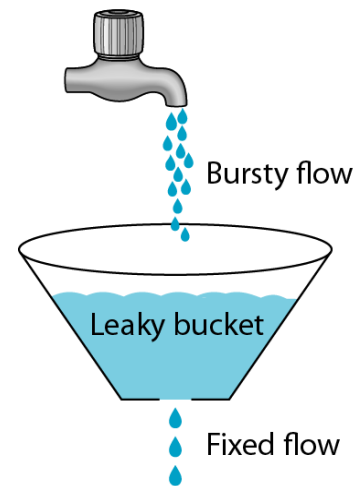


Traffic Shaping

- Traffic shaping is a mechanism to control the amount and the rate of the traffic sent to the network
- Two techniques can shape traffic:
 - leaky bucket
 - token bucket.

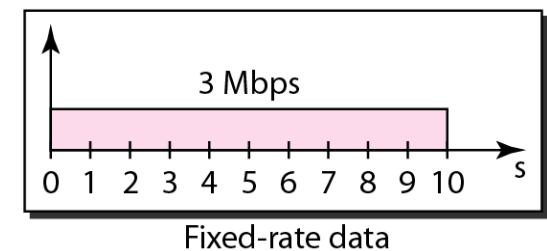
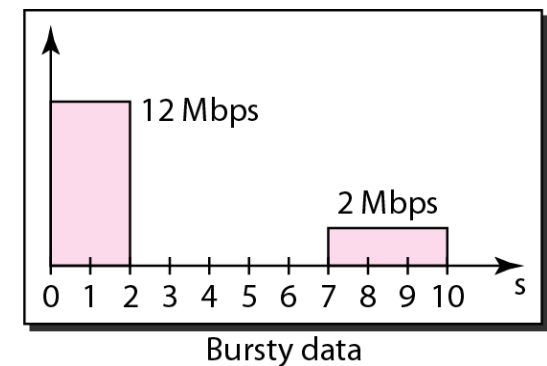
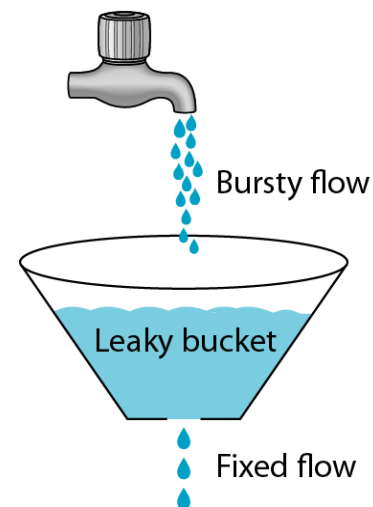
Leaky Bucket

- A bucket with small hole at the bottom, the water leaks at a constant rate as long as there is water in the bucket
- The rate at which the water leaks does not depend on the rate at which the water is input to the bucket unless the bucket is empty
- The input rate can vary, but the output rate remains constant

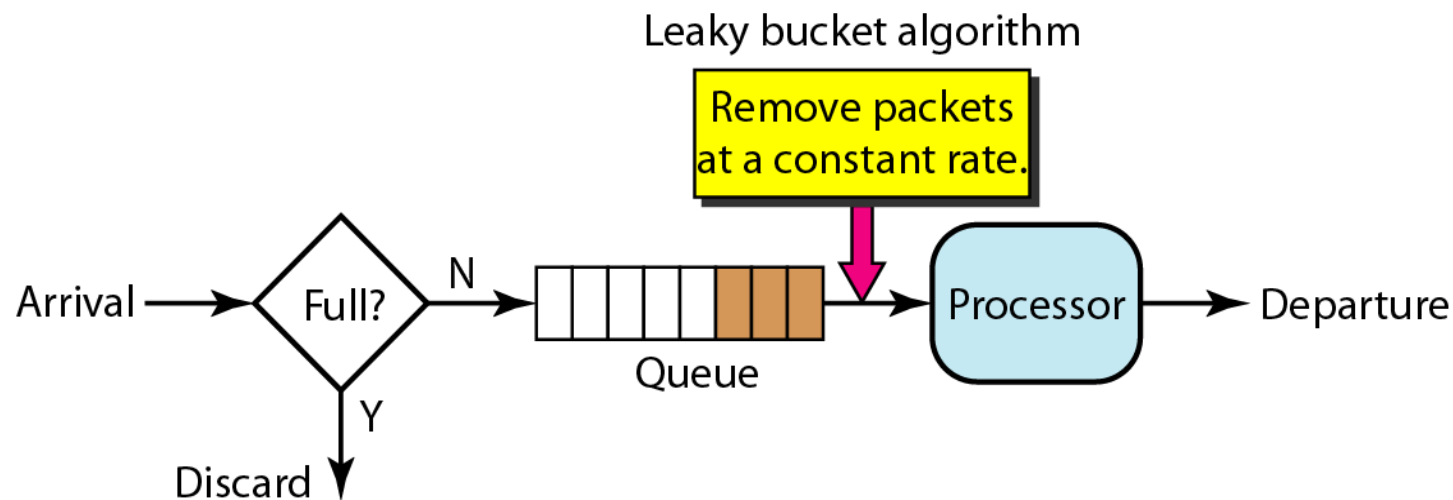


Leaky Bucket

- A bucket with small hole at the bottom, the water leaks at a constant rate as long as there is water in the bucket
- The rate at which the water leaks does not depend on the rate at which the water is input to the bucket unless the bucket is empty
- The input rate can vary, but the output rate remains constant
- Leaky bucket technique in networking
 - Bursty chunks are stored in the bucket and sent out at an average rate



Leaky Bucket Implementation



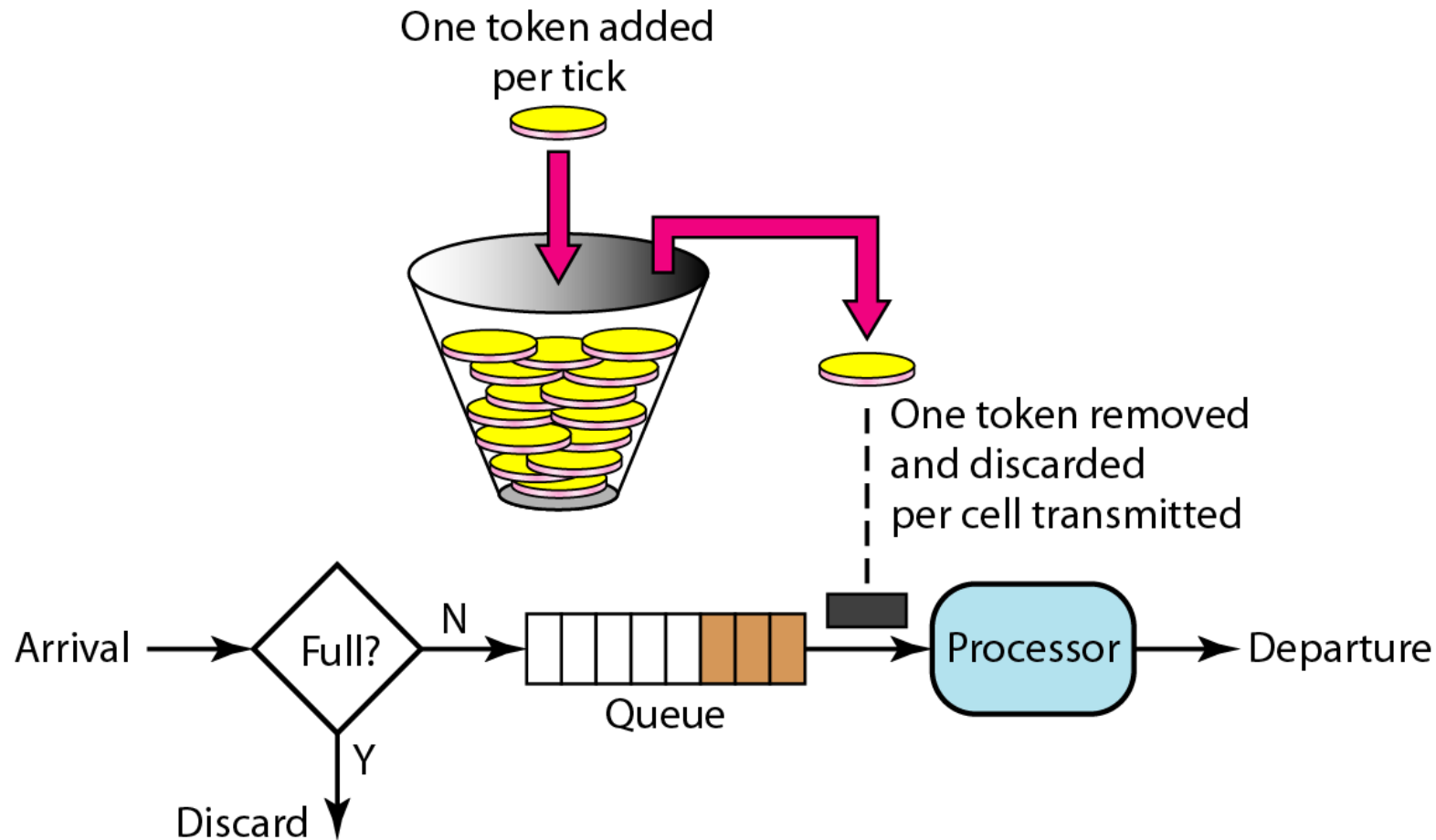
- A FIFO queue holds the packets
- If the traffic consists of fixed-size packets , the process removes a fixed number of packets from the queue at each tick of the clock
- If the traffic consists of variable-length packets, the fixed output rate must be based on the number of bytes or bits.



Token Bucket

- The time when the host was idle is not taken into account in leaky bucket
- For each tick of the clock, the system sends n tokens to the bucket
- The system removes one token for every cell (or byte) of data sent
 - For example, if n is 100 and the host is idle for 100 ticks, the bucket collects 10,000 tokens
 - Now the host can consume all these tokens in one tick with 10,000 cells, or the host takes 1000 ticks with 10 cells per tick
- In other words, the host can send bursty data as long as the bucket is not empty
- The token bucket can easily be implemented with a counter. The token is initialized to zero. Each time a token is added, the counter is incremented by 1. Each time a unit of data is sent, the counter is decremented by 1. When the counter is zero, the host cannot send data.

Token Bucket Implementation





Next : Application Layer